

## **Master prompt:**

I am building a complete Machine Learning + Streamlit project from scratch.

This is a major college project and I want it to be developed like a professional software project suitable for GitHub and my resume.

Treat this as a production-quality application.

Do NOT generate shortcut code.

Generate complete files only.

Generate one complete file at a time.

Whenever a previous file needs modification because of architecture improvements, regenerate that file completely.

Do NOT generate partial snippets.

-----

### PROJECT

Language Detection System

-----

### OBJECTIVE

Detect the language of input text with high accuracy.

-----

### DATASET

Kaggle Dataset

Language Detection

<https://www.kaggle.com/datasets/basilb2s/language-detection>

Dataset contains

17 languages

Approximately 10,337 records

-----

### TECH STACK

Python 3.11

Streamlit

Scikit-learn

Pandas

NumPy

Matplotlib

Plotly

Joblib

PyPDF2

python-docx

tqdm

-----

DO NOT USE

TensorFlow

PyTorch

Transformers

FastText

spaCy

NLTK

Paid APIs

-----

PROJECT STRUCTURE

Language-Detection-System/

data/

raw/

processed/

sample\_inputs/

notebooks/

01\_EDA.ipynb

02\_Model\_Comparison.ipynb

03\_Error\_Analysis.ipynb

src/

\_\_init\_\_.py

config.py

data\_loader.py

preprocess.py

feature\_engineering.py

model\_selection.py

trainer.py

evaluator.py

predictor.py

serializer.py

utils/

logger.py

helpers.py

pdf\_reader.py

docx\_reader.py

txt\_reader.py

models/

assets/

pages/

1\_Home.py

2\_Text\_Detection.py  
3\_File\_Detection.py  
4\_Batch\_Prediction.py  
5\_Model\_Performance.py  
6\_About.py  
app.py  
train\_model.py  
requirements.txt  
README.md  
.gitignore

---

## MACHINE LEARNING WORKFLOW

Raw Dataset

↓

Load Dataset

↓

Validate Dataset

↓

Preprocessing

↓

EDA

↓

Feature Engineering

↓

Create Character TF-IDF



Create Word TF-IDF



Combine using FeatureUnion



Build sklearn Pipeline



Compare Models



Naive Bayes



Logistic Regression



LinearSVC



Choose Best Baseline



GridSearchCV



Cross Validation



Evaluation



Classification Report



Confusion Matrix



Precision



Recall



F1



Save Pipeline



metrics.json



Streamlit



PREPROCESSING

Remove duplicates

Remove missing values

Trim spaces

Normalize whitespace

DO NOT

Remove punctuation

Remove accents

Remove language-specific symbols

Remove stopwords

Reason

Language detection depends on character sequences and scripts.

---

## FEATURE ENGINEERING

Character TF-IDF

```
analyzer="char"
```

```
ngram_range=(2,5)
```

Word TF-IDF

```
analyzer="word"
```

```
ngram_range=(1,2)
```

Combine both using FeatureUnion.

---

## MODEL COMPARISON

Train

MultinomialNB

LogisticRegression

LinearSVC

Compare baseline accuracy.

---

## FINAL MODEL

LinearSVC

---

## PIPELINE

Use sklearn Pipeline.

Pipeline

↓

FeatureUnion

↓

LinearSVC

Do NOT save separate vectorizer.

Save only

language\_detector.pkl

---

GRIDSEARCH

Use GridSearchCV

Optimize

classifier\_\_C

classifier\_\_loss

Use

5-fold StratifiedKfold

Optimize

f1\_macro

---

EVALUATION

Accuracy

Precision Macro

Precision Weighted

Recall Macro

Recall Weighted

F1 Macro

F1 Weighted

Classification Report

Confusion Matrix

Cross Validation Mean

Cross Validation Std

---

SAVE

language\_detector.pkl

metrics.json

---

## STREAMLIT FEATURES

Home

Text Detection

File Detection

Batch Prediction

Performance Dashboard

About

---

## FILE SUPPORT

TXT

PDF

DOCX

CSV

---

## CODING STANDARDS

Production quality

PEP-8

Google Docstrings

Type hints

Reusable functions

Exception handling

Logging

Single Responsibility Principle

Modular architecture

---

GIT

Every completed phase should end with

`git add .`

`git commit -m "Meaningful Commit"`

`git push`

---

IMPORTANT

Never generate the whole project at once.

Generate one COMPLETE FILE at a time.

Whenever you generate a file,

explain

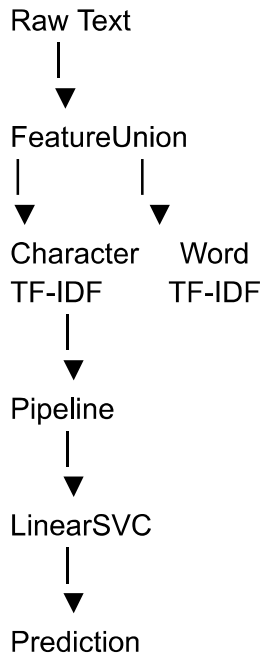
1. Why this file exists.

2. Workflow.

3. Testing.

4. Git commit.

Wait for my confirmation before generating the next file.



## # DEVELOPER SPECIFICATION

Follow these coding conventions throughout the project.

---

### GENERAL

Use Python 3.11

Follow PEP-8

Use Google Style Docstrings

Use Type Hints

Every file must be modular.

No duplicate code.

Every class should have only one responsibility.

Prefer classes over loose functions where appropriate.

---

### CONFIGURATION

Create

config.py

Contains

BASE\_DIR

DATA\_DIR

RAW\_DATASET

CLEAN\_DATASET

MODEL\_DIR

ASSETS\_DIR

TEST\_SIZE

RANDOM\_STATE

CHAR\_NGRAM\_RANGE

WORD\_NGRAM\_RANGE

Never hardcode paths elsewhere.

---

LOGGER

Use

`utils/logger.py`

Create one global logger.

Every module should use

```
from utils.logger import logger
```

Log

Loading

Training

Evaluation

Saving

Prediction

Errors

---

## DATA LOADER

### Class

DataLoader

### Methods

load\_dataset()

validate\_dataset()

### Responsibilities

Load CSV

Check dataset exists

Validate required columns

Return DataFrame

---

## PREPROCESSOR

### Class

Preprocessor

### Methods

clean\_text()

preprocess()

### Responsibilities

Drop duplicates

Drop missing values

Normalize whitespace

Trim spaces

Save cleaned dataset

---

## FEATURE ENGINEERING

### Class

FeatureEngineering

### Methods

get\_character\_vectorizer()

get\_word\_vectorizer()

get\_combined\_features()

### Libraries

TfidfVectorizer

FeatureUnion

Character TF-IDF

analyzer="char"

ngram\_range=(2,5)

Word TF-IDF

analyzer="word"

ngram\_range=(1,2)

---

## MODEL SELECTION

### Class

ModelSelection

### Method

get\_models()

Return Dictionary

{

"Naive Bayes": Pipeline(...),

"Logistic Regression": Pipeline(...),

"Linear SVC": Pipeline(...)

}

Libraries

Pipeline

LinearSVC

LogisticRegression

MultinomialNB

---

TRAINER

Rename

train.py

to

trainer.py

Class

Trainer

Responsibilities

Train/Test Split

Model Comparison

Choose Best Model

GridSearchCV

Cross Validation

Return Best Pipeline

Methods

split\_dataset()

compare\_models()

display\_results()

tune\_best\_model()

cross\_validate\_best\_model()

train()

Use

train\_test\_split

GridSearchCV

cross\_val\_score

StratifiedKFold

---

GRID SEARCH

Only tune

LinearSVC

Use

param\_grid

classifier\_\_C

classifier\_\_loss

Optimize

f1\_macro

---

EVALUATOR

Rename

evaluate.py

to

evaluator.py

Class

Evaluator

Methods

evaluate()

classification\_report()

confusion\_matrix()

save\_metrics()

Responsibilities

Accuracy

Precision

Recall

F1

Classification Report

Confusion Matrix

metrics.json

Libraries

accuracy\_score

precision\_score

recall\_score

f1\_score

classification\_report

confusion\_matrix

ConfusionMatrixDisplay

-----

SERIALIZER

Create

serializer.py

Class

ModelSerializer

Methods

save\_pipeline()

load\_pipeline()

save\_metrics()

Responsibilities

Save

language\_detector.pkl

Save

metrics.json

Use

joblib

json

-----

PREDICTOR

Rename

predict.py

to

predictor.py

Class

Predictor

Methods

predict\_text()

predict\_batch()

predict\_file()

Responsibilities

Load trained Pipeline

Return predicted language

Return confidence (decision\_function if supported)

-----

PIPELINE

Pipeline(

FeatureUnion(

Character TF-IDF

+

Word TF-IDF

)

↓

LinearSVC

)

Save entire Pipeline

Do NOT save vectorizer separately.

-----

METRICS

Evaluate

Accuracy

Precision Macro

Precision Weighted

Recall Macro

Recall Weighted

F1 Macro

F1 Weighted

Classification Report

Confusion Matrix

Cross Validation Mean

Cross Validation Std

-----

## NOTEBOOKS

01\_EDA.ipynb

Dataset Analysis

Missing Values

Duplicates

Language Distribution

Character Count

Word Count

02\_Model\_Comparison.ipynb

Compare

Naive Bayes

Logistic Regression

LinearSVC

03\_Error\_Analysis.ipynb

Misclassified Samples

Confusion Matrix

Failure Analysis

---

## STREAMLIT

Home

Text Detection

File Detection

Batch Prediction

Performance Dashboard

About

---

## UTILS

logger.py

helpers.py

pdf\_reader.py

docx\_reader.py

txt\_reader.py

---

## TRAIN MODEL

train\_model.py

Acts only as entry point.

Should call

Trainer

Evaluator

Serializer

No ML logic inside train\_model.py.

---

## PREDICTION

app.py

Should never train model.

Only load

language\_detector.pkl

Predict.

---

## CODE STYLE

Use

Classes

Private helper methods where required

Docstrings

Type hints

Meaningful variable names

No duplicated logic

Avoid global variables except config constants.

---

## GIT

Every completed phase

git add .

git commit -m "Meaningful Commit"

git push

---

## IMPORTANT

Whenever architecture improves,

regenerate the complete affected file.

Never provide partial modifications.

Keep the entire project internally consistent.

## Architecture Freeze

Do NOT change:

- Folder structure
- Tech stack
- TF-IDF approach
- Character n-grams
- Word n-grams
- FeatureUnion
- LinearSVC as the final expected model
- Pipeline-based serialization
- Python version
- Streamlit frontend
- Git workflow

If you believe an architectural improvement is necessary, explain why first and regenerate all affected files completely rather than making partial changes.

## **prompts**

i have been assigned to develop a project, the objective of the project is it should detect languages based on the input, i have been asked to develop the project completely with moderate using streamlit, i have the dataset from kaggle, since im new to this kind of projects, i dont know the workflow and no idea about training models etc all this stuff. So, choose the model and create a workflow clearly that makes the application robust and highly reliable with good percentage of detection, tell what kind of libraries would u use for it, what kind of model would u choose.

If u want any other details from my side ask me, after getting all the details, we can make a final master prompt

2.okay fine now give me the file structure, codes required, dependencies everything related

3.give me the commands to create the above file structure

4.okay i have done setting up environment, now give me the requirements that i have to download for this application, and make sure u use correct version

5.start with phase 1 and i want my project to connect with github and im unaware of git, so i want to implement using git in my project, so use git in this project for saving etc everything

## **Git workflow and commands:**

Create Feature



Test Feature



Git Commit



Push to GitHub



Start Next Feature

## Step 0 — Install Git

If you don't already have Git installed:

- Download Git for Windows from the official website: [Git for Windows](#)
- During installation, the default options are fine.

Verify the installation:

```
git --version
```

You should see something like:

```
git version 2.50.1.windows.1
```

---

## Step 1 — Initialize Git Repository

Inside your project folder:

```
git init
```

You'll see:

```
Initialized empty Git repository...
```

---

## Step 2 — Create **.gitignore**

Create or update **.gitignore** with the following:

```
# Virtual Environment
```

```
.venv/
```

```
# Python
```

```
__pycache__/
```

```
*.pyc
```

```
*.pyo
```

```
*.pyd
```

```
# Jupyter
```

```
.ipynb_checkpoints/
```

```
# VS Code
```

```
.vscode/
```

```
# Streamlit
.streamlit/secrets.toml
```

```
# Model Files
models/*.pkl
models/*.joblib
```

```
# Logs
*.log
```

```
# OS
.DS_Store
Thumbs.db
```

```
# Temporary Files
*.tmp
```

### Why ignore the model?

The trained model can become large. We can regenerate it anytime by running `train_model.py`, so it's usually better not to commit it. If you later want to share a trained model, we can use GitHub Releases or Git LFS.

---

## Step 3 — First Commit

Check status:

```
git status
```

Stage everything:

```
git add .
```

Commit:

```
git commit -m "Initial project structure"
```

---

## Step 4 — Create a GitHub Repository

We'll do this once you've made the first commit.

Create a repository on GitHub named:

```
language-detection-system
```

**Don't** initialize it with:

- README

- License
- .gitignore

because those already exist locally.

### **Git identity connection:**

git config --global user.name "Your Name"

git config --global user.email "[your\\_email@example.com](mailto:your_email@example.com)"

**Create a repo in github account in your mail and name it,**

Then connect it:

git remote add origin <https://github.com/<your-username>/language-detection-system.git>

Push:

git branch -M main

git push -u origin main

From then on, each time you finish a feature:

git add .

git commit -m "Meaningful description"

git push

### **Changing from old chat to new chat:**

- 1.okay now give me the entire things that u have done to this project, like models used, structure followed everthing and give me a doc, so i can paste into another chat
- 2.how do i tell the new chat about the methods that u have used, and all the libraries, workflows
- 3.this is okay, but what i mean is the name of the functions, metrix and other things that u have used in the previous codes, how do i tell the new chat about it?

### **CODEX commands after moving to codex from web chatgpt**

- 1.now im gonna give u a master prompt for this project after that i will mention what are the phases that has been completed in the project, and what are the functions, metrics used after that accordingly i will ask u to continue generating the codes for the remaining part of the project
- 2.as i have already mentioned above, this entire project is getting done in phases, so make sure u notify when to commit the changes
- 3.what is the next phase, what are things to be done in this phase, generate codes accordingly

### **Project documentation for preparation:**

the application is working good, okay now give me the entire workflow, what are the thing happening in this application, like what happens from the initial step, workflow among the files in the project, what file does what, how the pipeline is getting built, how are u comparing the models etc, everything pin to pin, a to z in this application, since i have developed using chatgpt i dont know what was actually happening behind, so if i need to explain it in an interview, i shouldnt be stalled, i need to get complete knowledge about this project in way such that, i have complete knowledge about what actually happening the project  
make a document, doesnt matter how long it is, include pin to pin and a to z entire process

I created the full project explanation document here:

[PROJECT\_EXPLANATION\_GUIDE.md](C:/Users/brawl/OneDrive/Desktop/Language-Detection-System/PROJECT\_EXPLANATION\_GUIDE.md)

It covers:

- complete end-to-end workflow
- what every important file does
- how training works
- how preprocessing works
- how `FeatureUnion` and TF-IDF work
- how model comparison happens
- why `LinearSVC` is the final model
- how `GridSearchCV` is used
- how evaluation works
- how model saving/loading works
- how Streamlit pages work
- how file prediction and batch prediction work
- interview-ready explanations
- likely interview questions and strong answers

For your next step, I strongly recommend:

1. read this file once completely
2. then open these files side by side while reading:
  - `[train_model.py](C:/Users/brawl/OneDrive/Desktop/Language-Detection-System/train_model.py)`
  - `[src/trainer.py](C:/Users/brawl/OneDrive/Desktop/Language-Detection-System/src/trainer.py)`
  - `[src/predictor.py](C:/Users/brawl/OneDrive/Desktop/Language-Detection-System/src/predictor.py)`
  - `[app.py](C:/Users/brawl/OneDrive/Desktop/Language-Detection-System/app.py)`
3. practice explaining the project in your own words from the document

If you want, I can do one more very useful step next: I can prepare a second document called something like `INTERVIEW_QA_GUIDE.md` containing:

- interviewer questions
- ideal answers
- simple explanations
- technical explanations
- follow-up questions they may ask on this project

From this prompt:

```
(.venv) C:\Users\brawl\OneDrive\Desktop\Language-Detection-System>
```

run:

```
streamlit run app.py
```

If that does not work, run:

```
python -m streamlit run app.py
```

If you want to retrain first, run:

```
python train_model.py
```

Then start the app with:

streamlit run app.py